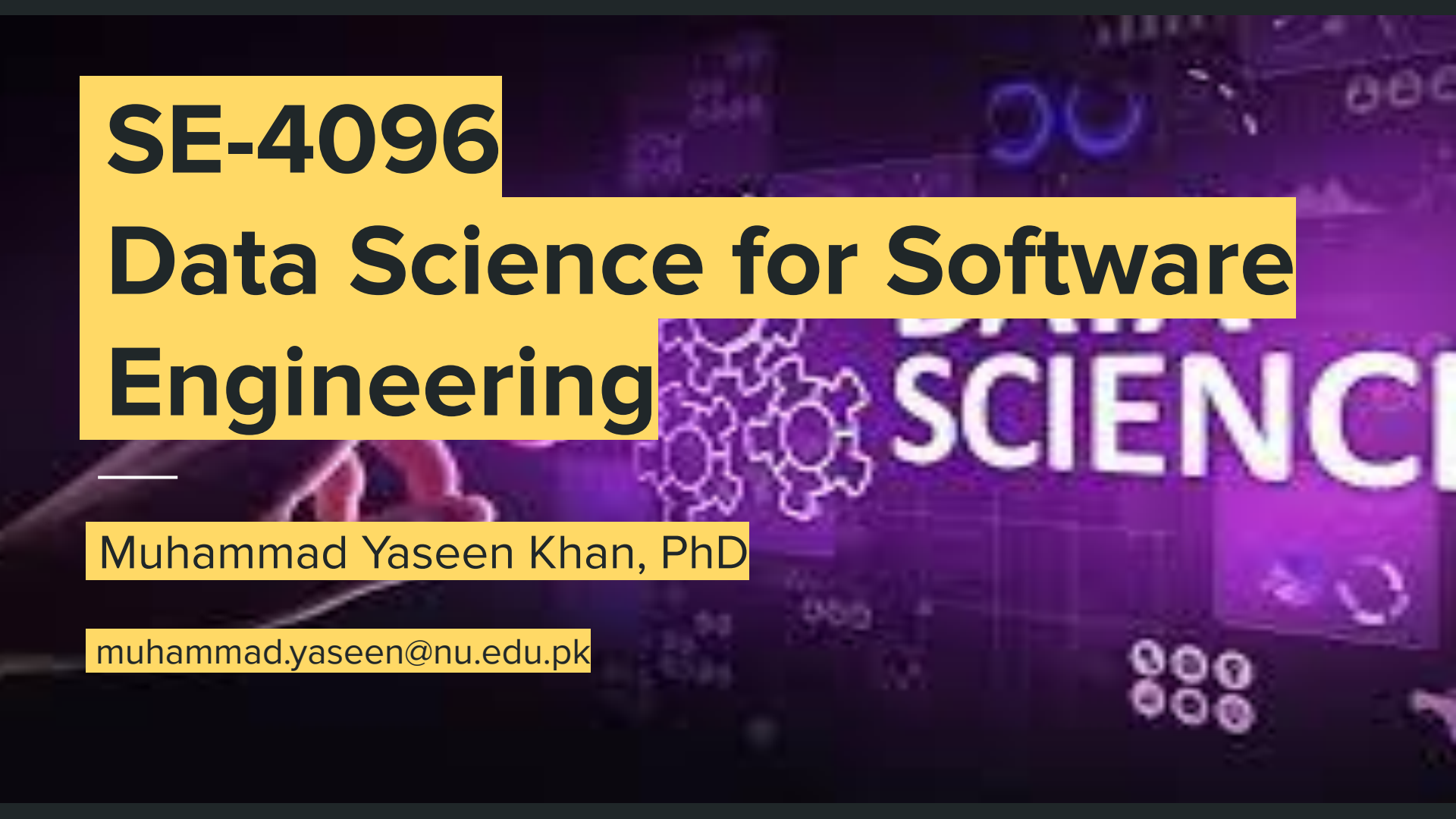




SE-4096

Data Science for Software Engineering

Muhammad Yaseen Khan, PhD

muhammad.yaseen@nu.edu.pk

**Data Science Workflow,
CRISP-DM, MSR,
Data Sources, Techniques &
Challenges**

Role of Data Science Workflow in SE

Role of Data Science Workflow in SE

1. Why software engineering problems need structured data workflows?

Software engineering problems involve *large*, *evolving*, and *noisy* data, so a **structured data workflow** ensures systematic analysis and reliable results.

Example: Without a workflow, bug data may be analyzed randomly; with CRISP-DM, we first define the goal (predict defects), then collect commits, clean data, build models, and evaluate results properly.

Role of Data Science Workflow in SE

2. Differences between traditional SE process models and data science workflows

Traditional SE models focus on building software features, while data science workflows focus on extracting insights and predictions from data.

Example: Waterfall defines steps like requirements → design → coding, whereas CRISP-DM defines steps like data understanding → preparation → modeling → evaluation.

Role of Data Science Workflow in SE

3. Iterative and exploratory nature of data-driven SE

Data-driven software engineering is iterative because insights improve through repeated analysis, experimentation, and refinement.

Example: After building a defect prediction model, poor results may require going back to feature selection or data cleaning instead of moving forward linearly.

Role of Data Science Workflow in SE

4. Importance of reproducibility and traceability in SE analytics

Reproducibility ensures results can be repeated, and traceability links insights back to the original software data and decisions.

Example: If a manager asks why a file was marked risky, reproducible scripts and traceable data links can show the commits, metrics, and model used.

Role of Data Science Workflow in SE

In nue:

**“Good software analytics is not just smart analysis,
it is structured, repeatable, and explainable.”**

CRISP-DM for Software Engineering

CRISP-DM for Software Engineering

CRISP-DM (Cross-Industry Standard Process for Data Mining) is a widely used, structured framework that defines the end-to-end process of conducting data science and data mining projects, from understanding the problem to deploying data-driven solutions.

CRISP-DM for Software Engineering

1. **Business Understanding** — Defining the software engineering problem and success criteria in terms of business or project goals.

Example: Predicting defect-prone files to reduce post-release bugs.

2. **Data Understanding** — Identifying, collecting, and exploring relevant software data to understand its structure, quality, and limitations.

Example: Analyzing Git commits and bug reports to see which files change most often.

3. **Data Preparation** — Cleaning, transforming, and selecting software data to make it suitable for modeling.

Example: Extracting code metrics (LOC, churn) and removing incomplete commit records.

CRISP-DM for Software Engineering

4. **Modeling** — Applying statistical or machine learning techniques to build predictive or analytical models.
Example: Training a classifier to predict whether a file is defective or not.
5. **Evaluation** — Assessing the model's performance and usefulness using appropriate metrics and validation methods.
Example: Evaluating a defect prediction model using Precision, Recall, and F1 score.
6. **Deployment** — Integrating the model or insights into real software engineering workflows for practical use.
Example: Using defect predictions inside a CI pipeline to flag risky commits.

Role of Data Science Workflow in SE

Summary:

“CRISP-DM provides a systematic and iterative framework for solving software engineering problems using data.”

Mining Software Repositories (MSR)

Mining Software Repositories (MSR)

What is MSR?

MSR is the process of extracting and analyzing data from software repositories to gain insights and improve software development.

Why MSR?

- Software projects generate large amounts of historical data
- MSR helps turn this data into **actionable knowledge**
- Supports evidence-based software engineering

Evolution of MSR

- Started with simple software metrics
- Evolved into a mature research area using:
 - Data science
 - Machine learning
 - Large-scale repository analysis

Mining Software Repositories (MSR)

MSR vs. Traditional Software Metrics

- Traditional metrics: static, limited, snapshot-based
- MSR: dynamic, historical, and behavior-oriented

Typical MSR Research Tasks

- **Defect prediction** (finding bug-prone code)
- **Code evolution analysis** (how software changes over time)
- **Developer behavior & collaboration analysis**
- **Technical debt identification**

Mining Software Repositories (MSR)

Key Challenges in MSR

- Large-scale and **heterogeneous data**
- Linking data across commits, issues, and developers
- **Noisy, incomplete**, and **biased** datasets
- Threats to:
 - Validity
 - Reproducibility

MSR Data Sources, Techniques & Tasks

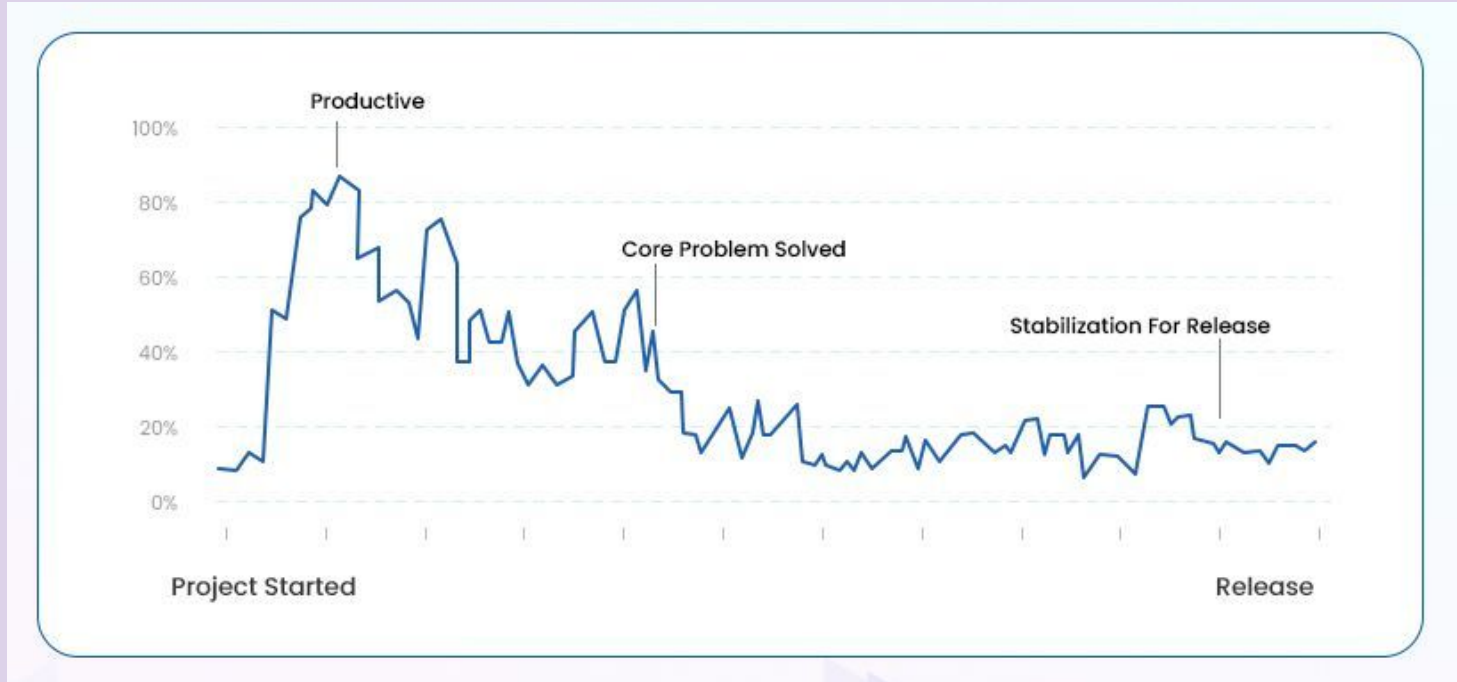
MSR Data Sources, Techniques & Tasks

1. Version Control Systems (VCS)

- Examples: **Git**, **SVN**
- Commits represent **units of change** in software
- Extractable data:
 - **Code churn** (lines added/deleted)
 - **Change frequency** of files
 - **Developer ownership** (who edits what)
 - **File evolution** over time

MSR Data Sources, Techniques & Tasks

code churn rate through the development of software



MSR Data Sources, Techniques & Tasks

Some More Examples

<https://githubnext.com/projects/repo-visualization/>

<https://datasciencecampus.github.io/visualising-github-commits/>

MSR Data Sources, Techniques & Tasks

2. Issue Tracking Systems

- Examples: **Jira**, **Bugzilla**, **GitHub Issues**
- Difference between:
 - **Bug reports** (errors/failures)
 - **Feature requests** (new functionality)
- Linking issues with commits for traceability
- Analysis of:
 - Severity
 - Priority
 - Resolution time

MSR Data Sources, Techniques & Tasks

3. CI/CD Systems

- Build and automated **test logs**
- **Test coverage** and test failure rates
- **Release frequency** and deployment stability
- Detection of:
 - Flaky tests
 - Unstable builds

MSR Data Sources, Techniques & Tasks

4. Code Review & Collaboration Platforms

- Pull requests and review discussions
- Review duration and reviewer participation
- Identifying bottlenecks in review process
- Impact of reviews on:
 - Code quality
 - Defect reduction

MSR Data Sources, Techniques & Tasks

5. Runtime and Operational Data

- Application logs and monitoring data
- System performance metrics (latency, errors)
- User behavior and usage patterns
- Post-deployment feedback:
 - Crash reports
 - User complaints (AppStore, Play Store)
 - Performance regressions

MSR Data Sources, Techniques & Tasks

6. Software Development Information

- Clustering and Topic Mining in:
 - Student Code Plag (turnitin and jPlag)
 - Overview and Scope Descriptions (Git README.md)
 - Software Discussion Forums
(Reddit, Tech Crunch, StackOverFlow, Quora, Medium)

Ethical Considerations in Software Data

Ethical Considerations in Software Data

1. Privacy of Developers and Teams

- **Developers as human subjects** — Developers are individuals whose work data reflects behavior and performance, so it must be treated ethically.
Example: Analyzing commit patterns can indirectly reveal working habits.
- **Risks of monitoring and surveillance** — Continuous tracking of developer activity can lead to stress, mistrust, and unfair evaluation.
Example: Ranking developers solely by number of commits.
- **Data anonymization and aggregation** — Removing personal identifiers and using group-level data to protect individual privacy.
Example: Reporting team averages instead of individual developer metrics.

Ethical Considerations in Software Data

1. Privacy of Developers and Teams

- **Developers as human subjects** — Developers are individuals whose work data reflects behavior and performance, so it must be treated ethically.
Example: Analyzing commit patterns can indirectly reveal working habits.
- **Risks of monitoring and surveillance** — Continuous tracking of developer activity can lead to stress, mistrust, and unfair evaluation.
Example: Ranking developers solely by number of commits.
- **Data anonymization and aggregation** — Removing personal identifiers and using group-level data to protect individual privacy.
Example: Reporting team averages instead of individual developer metrics.

Ethical Considerations in Software Data

2. Bias and Fairness in SE Analytics

- **Bias in open-source datasets** — Public repositories often over-represent popular projects and under-represent small or private ones.

Example: Models trained only on GitHub data may not generalize to enterprise software.

- **Misinterpretation of productivity metrics** — Simple metrics can give misleading conclusions about developer performance.

Example: More lines of code do not always mean more productive work.

- **Unequal impact of predictive models** — Predictions may unfairly target certain files, modules, or developers.

Example: Labeling all code by junior developers as high-risk.

Ethical Considerations in Software Data

3. Responsible Use of Metrics

- **Misuse of LOC and activity-based metrics** — Using raw activity counts as performance indicators can distort behavior.
Example: Developers writing unnecessary code to appear productive.
- **Goodhart's Law in software metrics** — When a metric becomes a target, it stops being a good measure.
Example: Optimizing commit count instead of code quality.
- **Avoiding harmful performance evaluations** — Metrics should support improvement, not punishment.
Example: Using defect data for coaching rather than blaming individuals.

Ethical Considerations in Software Data

4. Transparency and Explainability

- **Need for interpretable models** — Models should be understandable so stakeholders can trust and act on their results.

Example: Explaining why a file is predicted as defect-prone.

- **Communicating insights to stakeholders** — Analytical results must be presented in clear, non-technical language.

Example: Dashboards instead of raw model outputs.

- **Building trust in data-driven tools** — Trust increases when users understand how and why decisions are made.

Example: Showing factors contributing to risk predictions.

Lab:

- Introduction to
 - File I/O, JSON, Requests.py